TEL200 - Introduction to Robotics

Group 5

Yumi Lab Project

Authors

Andreas Brustad
Johannes Standal
Håkon Bekken

Norwegian University of Life Sciences

Contents

1 Abstract **2**

2 Introduction **3**
2.1 Yumi Application 3
2.2 Yumi Challenge 3

3 Method **4**
3.1 YuMi Application 4
3.2 YuMi Tic-Tac-Toe Challenge 6
3.3 CAD Models 7

4 Results **7**

5 Discussion **8**

6 Conclusions **9**

7 Bibliography **10**

1 Abstract

2 Introduction

2.1 Yumi Application

The goal for the yumi application part of the project was to enable the robot to manipulate objects. There are three blocks that shall be moved to their appropriate location, a button input for each block, and one emergency stop button. The blocks are all different shapes, and each have two spots on the table. The corresponding button input shall cause the robot to pick the block up and move it to its other spot. It should stop in its tracks at the push of the emergency stop button. There is also a fourth button that will move the robot back to its home position. To achieve this goal we will program the yumi lab robot using Robot Studio.

2.2 Yumi Challenge

The goal for the yumi challenge part of the project was to enable the robot to play tic tac toe. By placing paper on the table and making the robot grip a marker we could program it to draw shapes in the locations we want. Specifically, we made it draw an X at coordinate points in a 3×3 coordinate field. It's controlled through button inputs, one for the x-coordinate and one for the y-coordinate. Pressing the button will incrementally increase the corresponding value by 1. A third button is used to indicate that the desired values are reached, and start the related motion. To achieve this goal we will program the yumi lab robot using Robot Studio.

3 Method

3.1 YuMi Application

The control software for the dual-arm YuMi robot (IRB 14000) was developed in ABB's RAPID programming language using RobotStudio. The application implements a modular pick-and-place task that handles three distinct geometric objects — cube, cylinder, and triangular prism — triggered by external digital input signals. A central safety and recovery mechanism is provided through interrupt-driven emergency stop handling.

The program is contained in `MODULE Module1`. It defines a large number of `CONST robtarget` constants that store the taught positions for each object. These include safe approach points, gripping poses, lifting positions, transport poses, and drop-off locations. Each target is associated with the appropriate work object (`wobj`) — `Kube`, `sylinder`, `prisme` for pick-up phases, and typically `wobj0` for home and global movements. The positions were taught using the FlexPendant or RobotStudio's interface and include complete pose data (Cartesian position, orientation quaternion, robot configuration, and external axes values).

The main entry point is the `Main()` procedure, which first initializes the gripper system by calling `g_Init`. To avoid the common runtime error `ERR_ALRDYCNT` caused by connecting the same interrupt multiple times, an initialization guard is used:

```
IF NOT interrupts_setup THEN
    SkipWarn;
    CONNECT stop_emer WITH EmergencyStop;
    ISignalDI di_EmergencySituation, high, stop_emer;
    interrupts_setup := TRUE;
ENDIF
```

The program then enters an infinite loop that continuously polls three digital input signals (`di_cube`, `di_cylinder`, `di_prism`) and calls the corresponding handling procedure when a signal is high:

```
WHILE TRUE DO
    IF di_cube = 1 THEN Cube;
    ENDIF
    IF di_cylinder = 1 THEN syl;
    ENDIF
    IF di_prism = 1 THEN prism;
    ENDIF
ENDWHILE
```

Each object-specific procedure (`Cube`, `syl`, `prism`) follows a similar pick-and-place sequence:

1. Move to home position
2. Approach the pick-up area
3. Open gripper (`g_GripOut`) and wait
4. Descend to gripping pose
5. Close gripper (`g_GripIn`) and wait

6. Lift back to approach height
7. Transport to drop area
8. Descend to release pose
9. Open gripper and wait
10. Lift slightly
11. Return to home position

Most motions use `MoveJ` with speed `v500` and accuracy `fine` at critical points or `z100` during faster transfers. Wait times of 2 seconds after gripper commands ensure reliable mechanical action.

The emergency stop is implemented as a trap routine triggered on the rising edge of `di_EmergencySituation`:

```
TRAP EmergencyStop
    SkipWarn;
    StopMove;
    WaitUntil di_EmergencySituation = 0;
    StorePath;
    IF DInput(di_home) = 1 THEN
        ClearPath;
        MoveJ home, v500, fine, Servo\WObj:=wobj0;
        WaitTime 1;
        ExitCycle;
    ELSE
        RestoPath;
        StartMove;
    ENDIF
ENDTRAP
```

When the emergency input becomes active, motion stops immediately (`StopMove`). After the signal is released, the program checks `di_home`. If high, it clears the path context, moves to the safe `home` position, and executes `ExitCycle` to restart from the beginning of `Main()`. Otherwise it restores and resumes the interrupted path. This mechanism provides both immediate safety response and flexible recovery.

3.2 YuMi Tic-Tac-Toe Challenge

The Tic-Tac-Toe project was implemented on the dual-arm YuMi robot using two independent RAPID modules — one for the left arm and one for the right arm. Each arm operates autonomously but follows the same general logic: to select and draw an "X" in one of nine grid cells based on incoming digital signals.

Program structure

Both arms continuously poll three digital inputs:

- `di_cube`: increments the X coordinate
- `di_cylinder`: increments the Y coordinate
- `di_prism`: confirms execution

Two counters (`X` and `Y`) track the selected grid cell. Depending on the X–Y combination, the program calls one of several path procedures such as `Path_P00`, `Path_P11`, or `Path_P22`.

The human–robot grid is divided between the arms as follows:

Table 1: Grid cell assignment to robot arms

	X=0	**X=1**	**X=2**
Y=0	Right: P00	Right: P10	Left: P20
Y=1	Right: P01	Right: P11	Left: P21
Y=2	Right: P02	Left: P12	Left: P22

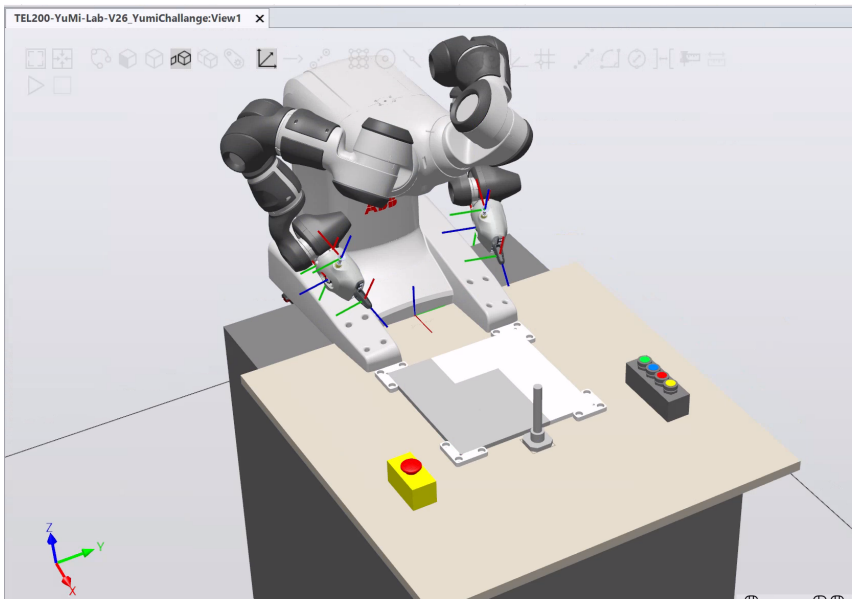


Figure 1: YumichallengeSetup in simulation and live.

Path execution

When a cell is selected, the corresponding path procedure executes a fixed sequence:

1. Move to the home position.
2. Move to `PenCenter` (safe pose above pen holder).
3. Open gripper (`g_GripOut`), descend to `PenPickDrop`, and close gripper (`g_GripIn`) to pick up the pen.
4. Move through a series of target points (`Target_10`, `Target_20`, etc.) to draw an "X".
5. Return to `PenCenter`, drop off the pen, and return to home.
6. Reset both `X` and `Y` to zero, ready for the next move.

Joint motions (`MoveJ`) are used for most positioning, while linear motions (`MoveL`) ensure smooth, straight drawing lines. Speeds are set to `v500` with fine or small blending zones (`z50`–`z100`) depending on path precision.

3.3 CAD Models

To support reliable pen handling, a custom CAD design was created for both the **marker holder** and the **pen adapter**. The parts were designed in Autodesk Fusion 360 and 3D-printed in PETG.

The holder fixes the pen in a repeatable position on the table, while the adapter ensures proper orientation and grip alignment with the robot's fingers. This modular approach improved drawing consistency and made tool pickup or drop-off fully repeatable between runs. Also a pen was designed in Autodesk that is identical in dimentions to the marker used in order to get correct point to floor distance.

A rendering of the final design is shown in Figure.

4 Results

5 Discussion

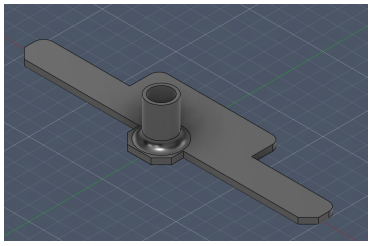
We started work on making the robot autonomously respond to user moves. By creating a python script to function as a tic-tac-toe AI, as well as a python script to serve as a bridge between the AI and the robot with TCP/IP, the robot can theoretically play without being manually controlled by the human. We envisioned the following logical flow:

User input for user move → Robot sends the user input as a packet to the python server → Python server calls the AI to get the robots move → Server returns robot move packet to robot → Robot follows the appropriate path for the received xy-coordinates

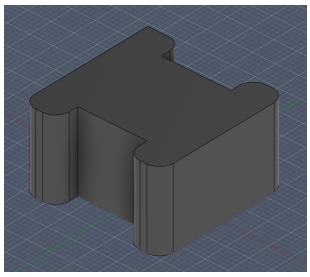
With this structure an autonomous robot tic-tac-toe player can be achieved, though it requires the robot to be connected via ethernet.

6 Conclusions

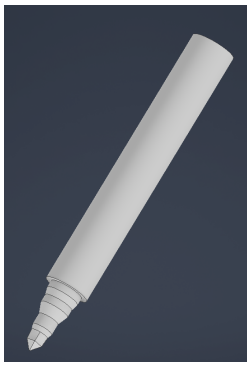
7 Bibliography



(a) The marker holder



(b) The pen adapter

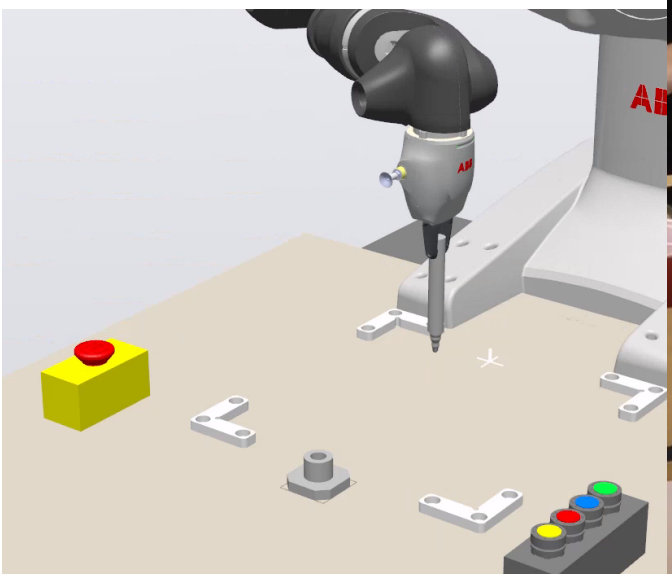


(c) Marker

(d) Marker and adapter

(e) Live demo

Figure 2: How the CAD models was implemented



(a) Simulation of drawing

(b) Drawing during real simulation